Representation and Storage of Digital Information

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

Introduction

Data vs. Information I

In the digital age, we are surrounded by data, but what does it mean?

- **Data:** Raw facts and figures without context (e.g., “38.5”).
- **Information:** Data that has been processed, organized, or structured to be meaningful (e.g., “The patient’s temperature is 38.5°C”).
- **Knowledge:** The ability to use information to make decisions.
- **Wisdom:** The integrated use of knowledge.

Data vs. Information II

To transform data into information, we need:

- **Structure:** A predefined format for the data.
- **Context:** Information about what the data represents.
- **Metadata:** “Data about data” (e.g., units, timestamps, origin).

The DIKW Pyramid

A model to represent the structural and functional relationships between data and wisdom.

- **Data:** The foundation (symbols).
- **Information:** Linked data (answers who, what, where, when).
- **Knowledge:** Applied information (answers how).
- **Wisdom:** Evaluated knowledge (answers why).

Why Representation Matters I

How we represent data affects every stage of the lifecycle:

- **Storage:** How much space it occupies (compression).
- **Speed:** How fast we can read or write it.
- **Interoperability:** Can different systems understand the same data?
- **Human Readability:** Can a human debug or edit the file easily?

Why Representation Matters II

- **Scalability:** Does the format work for 1GB? 1TB?
- **Security:** Can the data be easily tampered with?
- **Longevity:** Will the format be readable in 20 years?
- **Validation:** Can we check if the data is correct?

Data Categories

Classification of Data I

Data is generally classified into three categories based on its structure:

1. **Unstructured Data:** No predefined format.
2. **Semi-Structured Data:** Has some organizational properties but no strict schema.
3. **Structured Data:** Follows a strict, predefined model (usually tabular).

Classification of Data II

- **Choice:** The category depends on the nature of the data and how it will be used.
- **Migration:** We often extract structured data from unstructured sources (e.g., parsing logs).
- **Tools:** Each category requires different tools and expertise.

Unstructured Data

Characteristics of Unstructured Data

Unstructured data is the most common type of data in the world.

- **Examples:** Text documents, PDF files, emails, images, videos, logs.
- **Format:** Usually binary or raw text without a consistent internal structure.
- **Challenge:** It is difficult to search, index, and analyze using traditional tools.
- **Growth:** Estimated to be 80% of all enterprise data.

Processing Unstructured Data: Basic Tools I

Before using advanced tools, we use basic Unix utilities to explore text:

- `head` / `tail`: View the beginning or end of a file.
- `cat` / `less`: Print or browse the file content.
- `wc`: Count lines, words, and characters.
- `file`: Identify the type of data (e.g., text, image, binary).

Processing Unstructured Data: Basic Tools II

Example of exploration:

```
$ file data.log
$ wc -l data.log
$ head -n 5 data.log
```

Regular Expressions (Regex) I

To process unstructured text, we need a way to describe patterns.

- **Literal:** `abc` matches “abc”.
- **Wildcard:** `.` matches any character.
- **Quantifiers:**
  - `*`: 0 or more.
  - `+`: 1 or more.
  - `?`: 0 or 1.
  - `{n,m}`: between n and m.

Regular Expressions (Regex) II

- **Anchors:**
  - `^`: Start of line.
  - `$`: End of line.
- **Character Classes:**
  - `[a-z]`: Any lowercase letter.
  - `\d`: Any digit.
  - `\w`: Any word character (letters, numbers, underscore).
  - `\s`: Any whitespace.

Regex Example: Email Validation

A simplified pattern: `^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`

- `^[a-zA-Z0-9._%+-]+`: One or more characters at the start.
- `@`: The literal @ symbol.
- `[a-zA-Z0-9.-]+`: The domain name.
- `\.`: A literal dot.
- `[a-zA-Z]{2,}$`: The top-level domain (at least 2 letters) at the end.

Processing Unstructured Data: Logs I

Logs are a classic example of “mostly” unstructured data.

- Every action in a system generates a log entry.
- Often, they are just lines of text in a file.
- **Tools:** We use Unix “power tools” to extract information.
  - `grep`: Search for patterns.
  - `awk`: Process columns of text.
  - `sed`: Stream editor for text transformation.

Processing Unstructured Data: Logs II

Example of an Apache Access Log: `127.0.0.1 - - [20/Apr/2026:09:24:00 +0000] "GET /index.html HTTP/1.1" 200 612`

- It looks structured, but it's just a string.
- To know which IP visited most, we need to parse the text.
- `cat access.log | awk '{print $1}' | sort | uniq -c | sort -nr`

Advanced Log Processing: `awk`

`awk` is a complete programming language for text processing.

- It processes files line by line, split into fields ($1, $2, ...).
- **Usage:** `awk '$9 == 404 {print $7}' access.log`
- This command prints the URL ($7) for all requests that resulted in a 404 error ($9).

Stream Editing: sed

`sed` is used for transforming or filtering text.

- **Substitution:** `sed 's/old/new/g' file.txt`
- **Deletion:** `sed '1,5d' file.txt` (Delete lines 1 to 5).
- **Extraction:** `sed -n 's/.*ID:\([0-9]*\).*/\1/p' log.txt`
- `sed` is extremely fast and can process huge files that don’t fit in memory.

Semi-Structured Data

What is Semi-Structured Data? I

It does not reside in a fixed table but contains tags or markers to separate data elements.

- **Flexibility:** Can easily represent hierarchical and nested relationships.
- **Self-Describing:** The tags provide metadata within the file itself.
- **Common Formats:** JSON, XML, YAML.

What is Semi-Structured Data? II

- **Usage:**
  - Web APIs (REST/GraphQL).
  - Configuration files.
  - NoSQL databases (MongoDB, CouchDB).
  - Data exchange between heterogeneous systems.

JSON (JavaScript Object Notation) I

The “de facto” standard for modern web communication.

- **Format:** Key-Value pairs and Arrays.
- **Pros:** Lightweight, easy for humans to read, easy for machines to parse.
- **Data Types:** String, Number, Boolean, Null, Object, Array.

JSON (JavaScript Object Notation) II: Data Types

- **String:** `"mario"`
- **Number:** `123`, `12.3`
- **Boolean:** `true`, `false`
- **Null:** `null`
- **Object:** `{"key": "value"}`
- **Array:** `[1, 2, 3]`

JSON (JavaScript Object Notation) III: Example

```json
{
  "user": "mario",
  "id": 123,
  "active": true,
  "roles": ["admin", "teacher"],
  "address": {
    "city": "Aveiro",
    "zip": "3810"
  }
}
```

JSON Schema

To ensure a JSON file is correct, we use a **JSON Schema**.

- It defines the structure, required fields, and data types.
- Allows for automated validation before processing.

```
{
  "type": "object",
  "properties": {
    "user": {"type": "string"},
    "id": {"type": "integer", "minimum": 1}
  },
  "required": ["user", "id"]
}
```

XML (eXtensible Markup Language) I

An older, more verbose standard focused on document structure.

- **Format:** Nested tags `<tag>content</tag>`.
- **Pros:** Extremely robust, supports complex schemas (XSD), very strict.
- **Cons:** “Heavy” (lots of metadata overhead), harder for humans to read than JSON.

XML (eXtensible Markup Language) II: Attributes

Unlike JSON, XML can store data in **attributes**.

```
<user id="123" status="active">
  <name>mario</name>
</user>
```

- **Tags:** Good for hierarchical data.
- **Attributes:** Good for metadata about a tag.

XML (eXtensible Markup Language) III: Schema (XSD)

- **XSD (XML Schema Definition):** A way to define the rules for an XML file.
- It defines which tags are allowed, their order, and the type of data they contain.
- This allows for **automated validation** of the data.

XML (eXtensible Markup Language) IV: Namespaces

XML uses **Namespaces** to avoid tag collisions when combining different documents.

```
<root xmlns:h="http://www.w3.org/TR/html4/"
      xmlns:f="https://www.w3schools.com/furniture">
  <h:table>...</h:table>
  <f:table>...</f:table>
</root>
```

- `h:table` refers to an HTML table.
- `f:table` refers to a furniture table.

YAML (YAML Ain’t Markup Language) I

Designed to be the most human-friendly data format.

- **Format:** Uses indentation instead of braces or tags.
- **Pros:** Very clean, easy to write, supports comments.
- **Usage:** Configuration files (Docker, Kubernetes, GitHub Actions).
- **Warning:** Indentation is critical (like in Python).

YAML (YAML Ain’t Markup Language) II: Features

- **Lists:** Started with a dash `-`.
- **Comments:** Use #.
- **Multi-line strings:** Use `|` (keep newlines) or `>` (fold newlines).

- **Anchors and Aliases:** Reuse data within the same file.

```yaml
defaults: &base
  adapter: postgres
  host: localhost

development:
  <<: *base
  database: dev_db
```

YAML (YAML Ain’t Markup Language) III: Example

```yaml
user: mario
id: 123
active: true
roles:
  - admin
  - teacher
address:
  city: Aveiro
  zip: 3810
```

Structured Data

Characteristics of Structured Data

Data that fits perfectly into a table (rows and columns).

- **Schema:** Every row must have the same columns.
- **Efficiency:** Very fast to search and process using SQL or dataframes.
- **Examples:** CSV, TSV, Relational Databases (SQL).

CSV (Comma Separated Values) I

The simplest and most common format for tabular data exchange.

- **Format:** Each line is a record; fields are separated by a comma (,).
- **Pros:** Universal support (Excel, Python, R, Databases).
- **Cons:** No standard way to handle special characters or nested data.

CSV (Comma Separated Values) II: The “Problem”

What if a field contains a comma? `1,Mario Antunes,"Aveiro, Portugal",true`

- Fields with commas or quotes must be **quoted**.
- Different countries use different delimiters (e.g., `;` instead of `,`).
- **TSV (Tab Separated Values):** Uses tabs to avoid the comma problem.

CSV (Comma Separated Values) III: Encodings

CSV files often suffer from encoding issues.

- **UTF-8:** The modern standard (supports all languages).
- **ISO-8859-1 (Latin-1):** Common in older Windows/Excel files.
- **Problem:** Opening a Latin-1 file as UTF-8 results in "mojibake" (e.g., Ã¡ instead of á).

Binary Structured Formats (Briefly)

For Big Data, CSV is too slow and large.

- **Apache Parquet:** A columnar storage format. Efficient for reading specific columns.
- **Apache Avro:** A row-based format with a schema. Great for data streams.
- **Pros:** Compression, type safety, significantly faster than text.

Data Exploitation

Working with Data: `jq` I

`jq` is like `sed` for JSON data. It is essential for CLI engineers.

- It allows you to filter, transform, and format JSON from the command line.
- **Usage:** `cat data.json | jq '.user'`
- **Formatting:** `cat data.json | jq '.'` (Prettify).

Working with Data: `jq` II: Filtering

```
cat users.json | jq '.[] | select(.active == true) | .name'
```

1. `.[]`: Iterate over the array.
2. `select(...)`: Filter based on a condition.
3. `.name`: Extract only the name field.

Working with Data: yq

`yq` is the YAML version of `jq`.

- Often used to edit configuration files programmatically.
- `yq eval '.server.port = 8081' config.yml`
- Essential for CI/CD pipelines (GitHub Actions).

Data in Python I: CSV

```python
import csv

# Reading
with open('data.csv', 'r', encoding='utf-8') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['name'], row['age'])

# Writing
with open('out.csv', 'w', newline='', encoding='utf-8') as f:
    writer = csv.writer(f)
    writer.writerow(['id', 'name'])
    writer.writerow([1, 'mario'])
```

Data in Python II: JSON

```python
import json

# Parsing a string
obj = json.loads('{"id": 1, "name": "mario"}')

# Saving to a file
with open('data.json', 'w') as f:
    json.dump(obj, f, indent=4)
```

Data in Python III: YAML

```python
import yaml # Requires PyYAML

# Loading
with open('config.yml', 'r') as f:
    config = yaml.safe_load(f)

# Dumping
print(yaml.dump(config))
```

Serialization vs Deserialization I

- **Serialization:** Converting an object in memory (e.g., a Python dictionary) into a format that can be stored or transmitted (e.g., a JSON string).
- **Deserialization:** The reverse process: converting a string or file back into an object in memory.

Serialization vs Deserialization II

Why do we need this?

- **Persistence:** Save the state of a program to disk.
- **Transmission:** Send an object over the network (API).
- **Language Independence:** A Python program can send a JSON to a Java program.

Data Validation

Why Validate Data?

Bad data leads to bad results (“Garbage In, Garbage Out”).

- **Types:** Is it a number? Is it a date?
- **Ranges:** Is the age between 0 and 120?
- **Mandatory:** Is the email field present?
- **Relationships:** Does the department ID exist?

Validation with Pydantic I

Pydantic is the modern way to validate data in Python.

```
from pydantic import BaseModel, EmailStr, Field

class User(BaseModel):
    id: int
    name: str = Field(min_length=3)
    email: EmailStr
    age: int = Field(gt=0, lt=120)
```

Validation with Pydantic II

```
# Valid data
u = User(id=1, name="Mario", email="mario@ua.pt", age=30)

# Invalid data (raises ValidationError)
try:
    u2 = User(id=1, name="Ma", email="not-an-email", age=-5)
except Exception as e:
    print(e)
```

- Pydantic automatically converts types where possible (e.g., "123" to 123).

Summary

Summary

- **Categories:** Unstructured (logs), Semi-structured (JSON/YAML), Structured (CSV).
- **Patterns:** Use Regex for text processing and `awk`/`sed` for logs.
- **Choice:** Choose JSON for APIs, YAML for config, and CSV for bulk data.
- **Tools:** Master `jq` for JSON and `yq` for YAML.
- **Programmatic:** Use Python's standard libraries and **Pydantic** for robust data handling.
- **Validation:** Always validate data at the entry point of your system.